

TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



BÀI THI TIỂU LUẬN
MÔN: PHÁT TRIỂN ỨNG DỤNG IOT NHÓM 1

Đề tài:

**THIẾT KẾ HỆ THỐNG GIÁM SÁT NHIỆT ĐỘ VÀ ĐỘ ẨM DỰA
TRÊN ESP32**

Thừa Thiên Huế, năm 2026

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa đầy đủ
IoT	Internet of Things – Internet vạn vật
ESP32	Espressif Systems Module 32-bit
DHT22	Digital Humidity and Temperature – Cảm biến kỹ thuật số thế hệ 22
MQTT	Message Queuing Telemetry Transport
HTTP	Hypertext Transfer Protocol
API	Application Programming Interface
GPIO	General Purpose Input/Output
Wi-Fi	Wireless Fidelity
IDE	Integrated Development Environment
PCB	Printed Circuit Board
ADC	Analog-to-Digital Converter
PWM	Pulse Width Modulation
SDK	Software Development Kit
RTOS	Real-Time Operating System

MỤC LỤC

DANH MỤC CÁC TỪ VIẾT TẮT	2
MỤC LỤC	
MỞ ĐẦU	
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	7
1.1. Vi điều khiển ESP32	7
1.1.1. Tổng quan và lịch sử phát triển	7
1.1.2. Kiến trúc và thông số kỹ thuật	7
1.1.3. Khả năng kết nối Wi-Fi	8
1.1.4. Hệ sinh thái phát triển	8
1.2. Cảm biến DHT22	8
1.2.1. Giới thiệu chung	8
1.2.2. Nguyên lý hoạt động	9
1.2.3. Kết nối phần cứng	9
1.3. Nền tảng Blynk Cloud	9
1.3.1. Giới thiệu Blynk	9
1.3.2. Kiến trúc hệ thống Blynk	10
1.3.3. Virtual Pins và Datastreams	10
1.3.4. Auth Token và bảo mật	10
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG	11
2.1. Yêu cầu hệ thống	11
2.1.1. Yêu cầu chức năng	11
2.1.2. Yêu cầu phi chức năng	11
2.2. Sơ đồ khối hệ thống	11
2.3. Sơ đồ kết nối phần cứng	12
2.3.1. Danh sách linh kiện	12
2.3.2. Bảng kết nối chân	12
2.3.3. Hình ảnh mô phỏng trên Wokwi	13
2.4. Cấu hình diagram.json cho Wokwi	13
CHƯƠNG 3: LẬP TRÌNH VÀ CẤU HÌNH HỆ THỐNG	15
3.1. Cấu hình Blynk Cloud	15
3.1.1. Tạo Template trên Blynk	15
3.1.2. Tạo Datastreams	15
3.1.3. Tạo Web Dashboard	15

3.1.4. Tạo thiết bị và lấy Auth Token	16
3.2. Lập trình Firmware	16
3.2.1. Thư viện sử dụng	16
3.2.2. Mã nguồn hoàn chỉnh với giải thích	17
3.3. Luồng xử lý dữ liệu	20
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ	21
4.1. Hướng dẫn triển khai và kiểm thử	21
4.1.1. Chuẩn bị môi trường	21
4.1.2. Quy trình chạy mô phỏng trên Wokwi	21
4.1.3. Kiểm tra kết quả trên Blynk Dashboard	21
4.2. Kết quả thực nghiệm	22
4.2.1. Kết quả mô phỏng Wokwi	22
4.2.2. Kết quả trên Blynk Dashboard	22
4.3. Đánh giá hệ thống	23
4.3.1. Ưu điểm	23
4.3.2. Hạn chế và hướng khắc phục	23
4.4. Hướng phát triển	24
KẾT LUẬN	25
TÀI LIỆU THAM KHẢO	26

MỞ ĐẦU

1. Tính cấp thiết của đề tài

Trong bối cảnh cuộc cách mạng công nghiệp lần thứ tư (Industry 4.0) đang diễn ra mạnh mẽ trên toàn cầu, Internet vạn vật (IoT) đã trở thành một trong những nền tảng công nghệ cốt lõi, mang lại khả năng kết nối và tự động hóa chưa từng có. Việc giám sát các thông số môi trường như nhiệt độ và độ ẩm đóng vai trò then chốt trong nhiều lĩnh vực thiết yếu: từ sản xuất công nghiệp, bảo quản thực phẩm và dược phẩm, đến nông nghiệp thông minh và quản lý tòa nhà. Các hệ thống truyền thống thường đòi hỏi hạ tầng phức tạp, chi phí đầu tư cao và thiếu khả năng giám sát theo thời gian thực từ xa.

Sự xuất hiện của vi điều khiển ESP32 – một module tích hợp xử lý 32-bit, Wi-Fi và Bluetooth – cùng với các nền tảng đám mây IoT như Blynk đã tạo ra bước đột phá đáng kể. Bộ đôi này cho phép triển khai các hệ thống giám sát thông minh với chi phí tối thiểu nhưng vẫn đảm bảo độ tin cậy và khả năng mở rộng cao.

2. Mục tiêu nghiên cứu

- Nghiên cứu lý thuyết về vi điều khiển ESP32, cảm biến DHT22 và nền tảng Blynk Cloud.
- Thiết kế sơ đồ kết nối phần cứng và lập trình firmware hoàn chỉnh.
- Mô phỏng hệ thống trên công cụ Wokwi và kiểm chứng kết quả thực tế trên Blynk Dashboard.
- Đề xuất hướng phát triển và ứng dụng thực tế của hệ thống.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là hệ thống giám sát nhiệt độ và độ ẩm sử dụng vi điều khiển ESP32 kết hợp cảm biến DHT22. Phạm vi nghiên cứu tập trung vào thiết kế phần cứng, lập trình firmware, tích hợp nền tảng đám mây Blynk và kiểm thử mô phỏng thông qua công cụ Wokwi.

4. Phương pháp nghiên cứu

- Nghiên cứu tài liệu: Thu thập và phân tích các tài liệu kỹ thuật, datasheet linh kiện, tài liệu hướng dẫn của Blynk và ESP32.

- Thiết kế và lập trình: Xây dựng sơ đồ mạch, viết code Arduino, cấu hình Dashboard.
- Thực nghiệm và kiểm thử: Mô phỏng trên Wokwi, đánh giá kết quả dữ liệu trên Blynk Cloud.

5. Bố cục tiểu luận

Ngoài phần Mở đầu và Kết luận, tiểu luận được cấu trúc thành bốn chương chính:

Chương 1 trình bày cơ sở lý thuyết; Chương 2 trình bày thiết kế hệ thống; Chương 3 mô tả quá trình lập trình và cấu hình; Chương 4 trình bày kết quả kiểm thử và đánh giá.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Vi điều khiển ESP32

1.1.1. Tổng quan và lịch sử phát triển

ESP32 là vi điều khiển 32-bit do công ty Espressif Systems (Thượng Hải, Trung Quốc) phát triển và ra mắt vào năm 2016. Đây là thế hệ kế tiếp của ESP8266, được nâng cấp toàn diện về hiệu năng xử lý, khả năng kết nối không dây và tập tính năng ngoại vi. ESP32 nhanh chóng trở thành lựa chọn hàng đầu trong cộng đồng phát triển IoT toàn cầu nhờ sự cân bằng xuất sắc giữa hiệu suất, tiêu thụ năng lượng và giá thành [1].

1.1.2. Kiến trúc và thông số kỹ thuật

ESP32 được xây dựng trên lõi xử lý Xtensa LX6 dual-core với tần số hoạt động lên đến 240 MHz. Kiến trúc dual-core cho phép thực thi đa nhiệm hiệu quả: một nhân có thể đảm nhiệm việc xử lý giao thức Wi-Fi/Bluetooth, trong khi nhân còn lại tập trung xử lý logic ứng dụng.

Thông số	Giá trị
CPU	Xtensa dual-core 32-bit LX6, 240 MHz
RAM	520 KB SRAM
Flash	4 MB (tùy module)
Wi-Fi	802.11 b/g/n, 2.4 GHz
Bluetooth	BT 4.2 và BLE
GPIO	34 chân đa năng
ADC	12-bit, 18 kênh
DAC	8-bit, 2 kênh
UART/SPI/I2C	3/4/2 giao tiếp

PWM	16 kênh độc lập
Điện áp hoạt động	3.0V – 3.6V (3.3V tiêu chuẩn)
Nhiệt độ hoạt động	-40°C đến +85°C

1.1.3. Khả năng kết nối Wi-Fi

Một trong những điểm nổi bật nhất của ESP32 là module Wi-Fi tích hợp hỗ trợ chuẩn IEEE 802.11 b/g/n. ESP32 có thể hoạt động ở ba chế độ: Station (STA) – kết nối vào mạng Wi-Fi hiện có; Access Point (AP) – tạo điểm phát Wi-Fi; và chế độ kết hợp STA+AP đồng thời. Tốc độ truyền dữ liệu lên đến 150 Mbps cho phép ESP32 đáp ứng tốt các ứng dụng IoT yêu cầu băng thông cao [2].

Trong hệ thống này, ESP32 hoạt động ở chế độ Station, kết nối vào mạng Wi-Fi có sẵn để thiết lập kênh truyền dữ liệu với máy chủ Blynk Cloud thông qua giao thức TCP/IP.

1.1.4. Hệ sinh thái phát triển

ESP32 được hỗ trợ bởi hệ sinh thái phong phú: Arduino IDE với thư viện ArduinoESP32, ESP-IDF (framework chính thức của Espressif), MicroPython, và PlatformIO. Cộng đồng phát triển đông đảo đảm bảo nguồn tài liệu, thư viện và hỗ trợ kỹ thuật dồi dào, giảm đáng kể thời gian phát triển sản phẩm.

1.2. Cảm biến DHT22

1.2.1. Giới thiệu chung

DHT22 (còn gọi là AM2302) là cảm biến kỹ thuật số đo nhiệt độ và độ ẩm được sản xuất bởi AOSONG Electronics. So với phiên bản tiền nhiệm DHT11, DHT22 có độ chính xác vượt trội và dải đo rộng hơn, đáp ứng yêu cầu của các ứng dụng chuyên nghiệp.

Thông số	DHT11	DHT22
Dải đo nhiệt độ	0°C – 50°C	-40°C – +80°C
Sai số nhiệt độ	±2°C	±0.5°C
Dải đo độ ẩm	20% – 90% RH	0% – 100% RH

Sai số độ ẩm	$\pm 5\%$ RH	$\pm 2\%$ RH
Chu kỳ lấy mẫu	1 giây	0.5 giây
Điện áp hoạt động	3V – 5V	3V – 5V
Giao tiếp	Single-wire digital	Single-wire digital

1.2.2. Nguyên lý hoạt động

DHT22 tích hợp hai cảm biến vật lý riêng biệt trong một vỏ bọc: cảm biến nhiệt độ sử dụng NTC thermistor (điện trở nhiệt âm) và cảm biến độ ẩm sử dụng polymer capacitive (tụ điện polymer). Khi độ ẩm không khí thay đổi, hệ số điện môi của polymer thay đổi tương ứng, làm thay đổi điện dung của tụ và được chuyển đổi thành giá trị số.

Giao tiếp giữa DHT22 và vi điều khiển diễn ra qua một dây tín hiệu duy nhất (singlewire protocol). Quá trình truyền dữ liệu gồm 40 bit (16 bit nhiệt độ + 16 bit độ ẩm + 8 bit checksum) và diễn ra theo trình tự: vi điều khiển gửi tín hiệu Start, DHT22 phản hồi ACK, sau đó truyền 40 bit dữ liệu. 8 bit checksum là tổng của 4 byte trước để đảm bảo tính toàn vẹn dữ liệu [3].

1.2.3. Kết nối phần cứng

DHT22 có 4 chân: VCC (3.3V–5V), DATA, NC (không kết nối), và GND. Chân DATA kết nối với GPIO của ESP32 thông qua điện trở kéo lên (pull-up resistor) 4.7k Ω đến 10k Ω để đảm bảo mức logic HIGH khi đường truyền ở trạng thái rảnh. Trong Wokwi, điện trở pull-up đã được tích hợp nội bộ nên không cần kết nối thêm.

1.3. Nền tảng Blynk Cloud

1.3.1. Giới thiệu Blynk

Blynk là nền tảng IoT đám mây được thành lập năm 2012, cung cấp giải pháp toàn diện cho việc kết nối thiết bị phần cứng với ứng dụng di động và web dashboard. Blynk 2.0 (phiên bản hiện tại) được kiến trúc lại hoàn toàn so với phiên bản cũ, tập trung vào khả năng quản lý nhiều thiết bị, bảo mật nâng cao và tích hợp doanh nghiệp [4].

1.3.2. Kiến trúc hệ thống Blynk

Kiến trúc Blynk bao gồm ba thành phần chính liên kết chặt chẽ:

- Blynk.Cloud Server: Máy chủ trung tâm xử lý kết nối, định tuyến dữ liệu, lưu trữ lịch sử và cung cấp API. Blynk sử dụng giao thức độc quyền dựa trên TCP/TLS, đảm bảo bảo mật và độ trễ thấp.
- Blynk Library (Firmware): Thư viện tích hợp vào firmware của thiết bị (ESP32, Arduino...), xử lý việc kết nối Wi-Fi, xác thực Auth Token và truyền/nhận dữ liệu qua Virtual Pins.
- Blynk Web Dashboard / Mobile App: Giao diện người dùng để giám sát và điều khiển thiết bị theo thời gian thực, tùy chỉnh widget và xem lịch sử dữ liệu.

1.3.3. Virtual Pins và Datastreams

Blynk sử dụng khái niệm Virtual Pins (V0 đến V255) làm kênh truyền dữ liệu trừu tượng giữa thiết bị và dashboard. Mỗi Virtual Pin tương ứng với một Datastream – một luồng dữ liệu có kiểu dữ liệu, đơn vị và phạm vi giá trị được định nghĩa sẵn. Cơ chế này tách biệt hoàn toàn lớp phần cứng (GPIO vật lý) khỏi lớp ứng dụng, tạo nên sự linh hoạt cao trong thiết kế hệ thống.

1.3.4. Auth Token và bảo mật

Mỗi thiết bị trong Blynk được cấp một Auth Token duy nhất – chuỗi ký tự 32 ký tự dùng để xác thực kết nối. Thiết bị chỉ có thể giao tiếp với server khi cung cấp đúng Auth Token. Kênh truyền được mã hóa TLS 1.2, đảm bảo an toàn thông tin trong quá trình truyền dữ liệu.

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

2.1. Yêu cầu hệ thống

2.1.1. Yêu cầu chức năng

- Đọc nhiệt độ và độ ẩm môi trường từ cảm biến DHT22 với chu kỳ 1 giây.
- Gửi dữ liệu nhiệt độ và độ ẩm lên Blynk Cloud theo thời gian thực.
- Hiển thị thời gian hoạt động (uptime) trên màn hình 7-segment TM1637.
- Hỗ trợ điều khiển LED thông qua cả Blynk Dashboard (V0) và nút nhấn vật lý.
- Đồng bộ trạng thái LED giữa điều khiển vật lý và giao diện Blynk.

2.1.2. Yêu cầu phi chức năng

- Kết nối Wi-Fi ổn định và tự động kết nối lại khi mất mạng.
- Thời gian phản hồi từ cảm biến đến Dashboard dưới 2 giây.
- Hoạt động liên tục 24/7 không cần can thiệp thủ công.
- Mã nguồn có chú thích rõ ràng, dễ bảo trì và mở rộng.

2.2. Sơ đồ khối hệ thống

Hệ thống được thiết kế theo kiến trúc phân lớp rõ ràng, gồm ba lớp chính:

Lớp thu thập dữ liệu (Data Acquisition Layer):

Cảm biến DHT22 đọc nhiệt độ và độ ẩm môi trường. Nút nhấn vật lý ghi nhận tín hiệu điều khiển từ người dùng. Các tín hiệu này được đưa vào ESP32 để xử lý.

Lớp xử lý và truyền dữ liệu (Processing & Communication Layer):

ESP32 đóng vai trò trung tâm: thu nhận dữ liệu từ cảm biến, điều khiển thiết bị ngoại vi (LED, TM1637), và thiết lập kết nối Wi-Fi để truyền dữ liệu lên Blynk Cloud qua giao thức TCP/TLS.

Lớp hiển thị và điều khiển (Presentation & Control Layer):

Blynk Web Dashboard cung cấp biểu đồ thời gian thực, gauge hiển thị giá trị tức thời và nút bật/tắt LED. Màn hình TM1637 hiển thị trực tiếp thời gian hoạt động tại thiết bị.

2.3. Sơ đồ kết nối phần cứng

2.3.1. Danh sách linh kiện

STT	Linh kiện	Số lượng	Chức năng
1	ESP32 DevKit C v4	1	Vi điều khiển chính, xử lý và kết nối WiFi
2	DHT22 (AM2302)	1	Cảm biến nhiệt độ và độ ẩm
3	TM1637 4-digit Display	1	Hiển thị thời gian hoạt động
4	LED xanh	1	Đèn trạng thái hệ thống
5	Nút nhấn (Push Button)	1	Điều khiển vật lý
6	Điện trở 220Ω	1	Hạn dòng cho LED

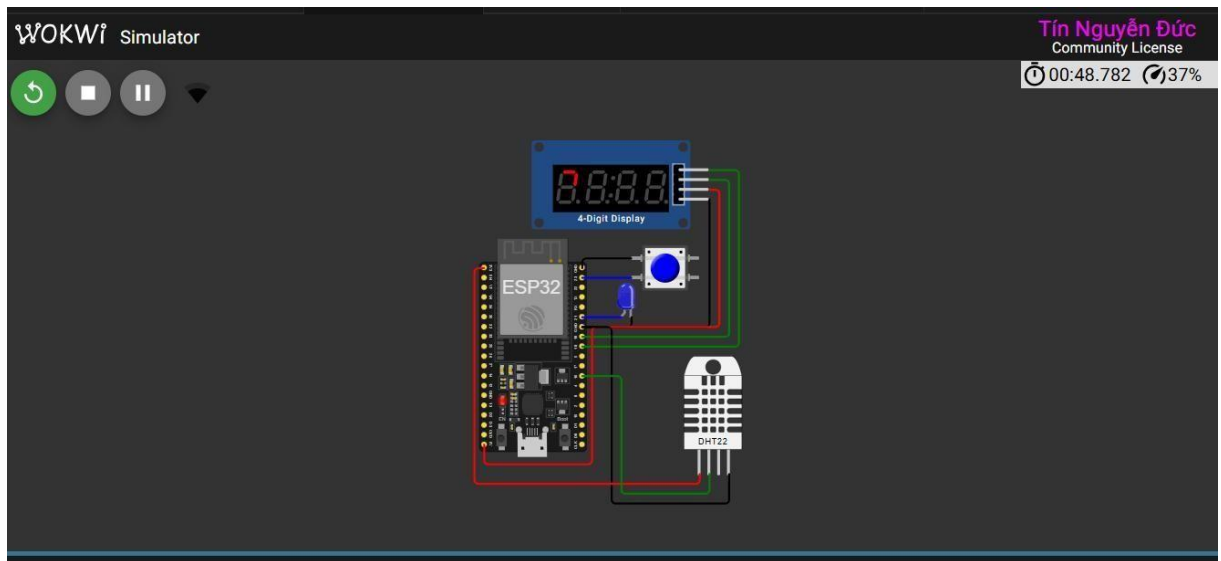
2.3.2. Bảng kết nối chân

Linh kiện	Chân linh kiện	Chân ESP32	Màu dây
DHT22	VCC	3.3V	Đỏ
DHT22	DATA	GPIO 16	Xanh lá
DHT22	GND	GND	Đen
TM1637	VCC	5V	Đỏ
TM1637	GND	GND	Đen
TM1637	CLK	GPIO 18	Xanh lá
TM1637	DIO	GPIO 19	Xanh lá
LED	Anode (+)	GPIO 21	Xanh dương

LED	Cathode (-)	GND	Đen
Nút nhấn	Chân 1	GND	Đen
Nút nhấn	Chân 2	GPIO 23	Xanh dương

2.3.3. Hình ảnh mô phỏng trên Wokwi

Sơ đồ kết nối thực tế trên công cụ mô phỏng Wokwi được minh họa trong hình dưới đây:



Hình 2.1: Sơ đồ mạch điện mô phỏng trên Wokwi

2.4. Cấu hình diagram.json cho Wokwi

File diagram.json sau đây mô tả toàn bộ sơ đồ kết nối có thể dán trực tiếp vào Wokwi Simulator:

```
{
  "version": 1,
  "author": "nguyenductin",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": -9.6, "left": -225.56, "attrs": {} },
    { "type": "wokwi-pushbutton", "id": "btn1", "top": -3.4, "left": -76.8, "attrs": { "color": "blue", "xray": "1" } },
    { "type": "wokwi-tm1637-7segment", "id": "sevseg1", "top": -105.64, "left": -175.37, "attrs": { "color": "red" } },
    { "type": "wokwi-led", "id": "led1", "top": 25.2, "left": -101.4, "attrs": { "color": "blue", "flip": "1" } },
    { "type": "wokwi-dht22", "id": "dht22", "top": 10.5, "left": -101.4, "attrs": {} }
  ]
}
```

```
{ "type": "wokwi-dht22", "id": "dht1", "top": 105.9,
"left": -24.6, "attrs": {} }
],
"connections": [
  ["esp:TX", "$serialMonitor:RX", "", []],
  ["esp:RX", "$serialMonitor:TX", "", []],
  ["btn1:1.1", "esp:GND.2", "black", ["h-48"]],
  ["btn1:2.1", "esp:23", "blue", ["h0"]],
  ["esp:19", "sevseg1:DIO", "green", ["h144", "v-9.6"]],
  ["sevseg1:CLK", "esp:18", "green", ["h28.8", "v172.8"]],
  ["sevseg1:GND", "esp:GND.3", "black", ["h0", "v124.8"]],
  ["led1:C", "esp:GND.3", "black", ["v9.6", "h0.4"]],
  ["led1:A", "esp:21", "blue", ["v0"]],
  ["sevseg1:VCC", "esp:5V", "red", ["h9.6", "v134.4", "h-
124.8", "v134.4", "h-105.6"]],
  ["dht1:GND", "esp:GND.3", "black", ["v28.8", "h-115.2", "v-
163.2"]],
  ["dht1:VCC", "esp:3V3", "red", ["v9.6", "h-220.8", "v-
201.6"]],
  ["dht1:SDA", "esp:16", "green", ["v19.2", "h-86.3",
"v105.6"]]
]
}
```

CHƯƠNG 3: LẬP TRÌNH VÀ CẤU HÌNH HỆ THỐNG

3.1. Cấu hình Blynk Cloud

3.1.1. Tạo Template trên Blynk

Để bắt đầu sử dụng Blynk, người dùng cần thực hiện theo trình tự sau:

1. Truy cập địa chỉ <https://blynk.cloud> và đăng ký tài khoản miễn phí (hoặc đăng nhập).
2. Tại giao diện chính, chọn Developer Zone → My Templates → New Template.
3. Điền thông tin: Template Name (ví dụ: "nguyenductin"), Hardware chọn "ESP32", Connection Type chọn "WiFi".
4. Nhấn Done để tạo Template. Ghi lại BLYNK_TEMPLATE_ID và BLYNK_TEMPLATE_NAME được cấp.

3.1.2. Tạo Datastreams

Datastream là kênh dữ liệu kết nối giữa thiết bị và Dashboard. Cần tạo bốn Datastream:

Virtual Pin	Tên Datastream	Kiểu dữ liệu	Min	Max	Đơn vị	Chức năng
V0	den_xanh	Integer	0	1	—	Điều khiển LED ON/OFF
V1	nhiet_do	Double	-40	80	°C	Giá trị nhiệt độ
V2	do_am	Double	0	100	%	Giá trị độ ẩm
V3	thoi_gian_hd	Integer	0	99999	s	Thời gian hoạt động

3.1.3. Tạo Web Dashboard

Sau khi tạo Datastreams, vào tab Web Dashboard và thêm các widget:

- Switch Widget: Liên kết với V0 (den_xanh) để bật/tắt LED từ xa.
- Gauge Widget (Nhiệt độ): Liên kết V1, cài đặt min -40, max 80, màu gradient từ xanh đến đỏ.

- Gauge Widget (Độ ẩm): Liên kết V2, cài đặt min 0, max 100, màu xanh lá.
- Chart Widget (Biểu đồ nhiệt độ): Liên kết V1, dạng line chart, hiển thị lịch sử 1 giờ.
- Chart Widget (Biểu đồ độ ẩm): Liên kết V2, dạng line chart, màu xanh dương.
- Value Display (Thời gian hoạt động): Liên kết V3, hiển thị số nguyên dạng số lớn.

3.1.4. Tạo thiết bị và lấy Auth Token

5. Vào My Devices → Add New Device → From Template, chọn template vừa tạo.
6. Hệ thống tự động sinh Auth Token (chuỗi 32 ký tự) – sao chép và lưu lại, không chia sẻ công khai.
7. Auth Token này sẽ được điền vào hằng số BLYNK_AUTH_TOKEN trong firmware.

3.2. Lập trình Firmware

3.2.1. Thư viện sử dụng

Thư viện	Phiên bản	Chức năng
Blynk (BlynkSimpleEsp32)	1.3.2+	Kết nối và truyền dữ liệu với Blynk Cloud
DHT sensor library	1.4.4+	Đọc dữ liệu từ cảm biến DHT22
TM1637Display	1.2.0+	Điều khiển màn hình 7-segment TM1637
WiFi (built-in ESP32)	—	Quản lý kết nối Wi-Fi
BlynkTimer	—	Đặt lịch thực thi hàm định kỳ

3.2.2. Mã nguồn hoàn chỉnh với giải thích

Dưới đây là toàn bộ mã nguồn firmware được viết theo chuẩn Arduino IDE, có chú thích chi tiết cho từng đoạn:

a) Phần khai báo và khởi tạo

```
// ===== CẤU HÌNH BLYNK =====  
// BLYNK_TEMPLATE_ID: Mã định danh duy nhất của Template trên  
Blynk Cloud  
#define BLYNK_TEMPLATE_ID "TMPL68k02dsRr"  
#define BLYNK_TEMPLATE_NAME "nguyenductin"  
// Auth Token: Mã xác thực thiết bị, cần giữ bí mật  
#define BLYNK_AUTH_TOKEN "dwjW7GFilJsaConmIM9oGd6jDBeJmwz_"  
  
// Cho phép in thông báo debug ra Serial Monitor
```

```
#define BLYNK_PRINT Serial

// ===== INCLUDE THƯ VIỆN =====
#include <Arduino.h>
#include <WiFi.h> // Thư viện Wi-Fi tích hợp ESP32
#include <WiFiClient.h> // Client TCP cho kết nối Blynk
#include <BlynkSimpleEsp32.h> // Thư viện Blynk cho ESP32
#include <DHT.h> // Thư viện đọc cảm biến DHT
#include <TM1637Display.h> // Thư viện màn hình 7-segment

// ===== THÔNG TIN WI-FI =====
// Wokwi cung cấp mạng ảo "Wokwi-GUEST" không cần mật khẩu
char ssid[] = "Wokwi-GUEST"; char pass[] = "";

// ===== ĐỊNH NGHĨA CHÂN (PIN) =====
#define DHTPIN 16 // GPIO16: chân DATA của DHT22
#define DHTTYPE DHT22 // Loại cảm biến: DHT22
#define LED_PIN 21 // GPIO21: điều khiển LED
#define BTN_PIN 23 // GPIO23: đọc nút nhấn (INPUT_PULLUP)
#define CLK 18 // GPIO18: chân CLK của TM1637
#define DIO 19 // GPIO19: chân DIO của TM1637

// ===== KHỞI TẠO ĐỐI TƯỢNG =====
DHT dht(DHTPIN, DHTTYPE); // Đối tượng cảm biến DHT22
TM1637Display display(CLK, DIO); // Đối tượng màn hình TM1637
BlynkTimer timer; // Bộ hẹn giờ Blynk

// ===== BIẾN TOÀN CỤC =====
int uptime = 0; // Đếm thời gian hoạt động (giây)
bool isDeviceOn = false; // Trạng thái thiết bị (ON/OFF)
int lastBtnState = HIGH; // Trạng thái nút nhấn trước đó
```

b) Hàm xử lý lệnh từ Blynk (V0)

```
// BLYNK_WRITE(V0): Hàm callback được gọi khi Blynk gửi
// dữ liệu xuống Virtual Pin V0 (nút bật/tắt trên Dashboard)
BLYNK_WRITE(V0) {
    // param.asInt() đọc giá trị: 1 = ON, 0 = OFF
    isDeviceOn = param.asInt();
    // Điều khiển LED vật lý theo trạng thái nhận được
    digitalWrite(LED_PIN, isDeviceOn ? HIGH : LOW);
    // Tắt màn hình khi thiết bị OFF if
    (!isDeviceOn) display.clear();
}
```

c) Hàm gửi dữ liệu lên Blynk Cloud

```
void sendDataToBlynk() {
    // --- Cập nhật uptime và gửi lên V3 ---
    uptime++; // Tăng bộ đếm mỗi lần hàm được gọi (1 giây)
    Blynk.virtualWrite(V3, uptime); // Gửi giá trị lên Virtual
    Pin V3

    // Hiển thị uptime lên TM1637 nếu thiết bị đang ON if
    (isDeviceOn) { display.showNumberDec(uptime); // Hiển
    thị số thập phân }

    // --- Đọc dữ liệu từ DHT22 --- float t =
    dht.readTemperature(); // Đọc nhiệt độ (°C) float h =
    dht.readHumidity(); // Đọc độ ẩm (%)

    // Kiểm tra lỗi đọc cảm biến (isnan = "is not a number") if
    (!isnan(t) && !isnan(h)) {
        Blynk.virtualWrite(V1, t); // Gửi nhiệt độ lên V1
        Blynk.virtualWrite(V2, h); // Gửi độ ẩm lên V2
        Serial.print("Nhiệt độ: "); Serial.print(t);
        Serial.print(" - Độ ẩm: "); Serial.println(h);
    } else {
        Serial.println("Lỗi đọc cảm biến DHT!");
    }
}
```

d) Hàm kiểm tra nút nhấn vật lý void

```
checkPhysicalButton() { int
btnState = digitalRead(BTN_PIN);
    // Phát hiện cạnh xuống: trạng thái HIGH -> LOW if
    (lastBtnState == HIGH && btnState == LOW) {
        delay(50); // Debounce: chờ 50ms để loại nhiễu if
        (digitalRead(BTN_PIN) == LOW) { isDeviceOn =
        !isDeviceOn; // Đảo trạng thái
        digitalWrite(LED_PIN, isDeviceOn ? HIGH : LOW); if
        (!isDeviceOn) display.clear();
        // Đồng bộ trạng thái mới lên Blynk Dashboard
        Blynk.virtualWrite(V0, isDeviceOn ? 1 : 0);
    } } lastBtnState = btnState; // Lưu trạng thái
    hiện tại

}
```

e) Hàm setup() và loop()

```
void setup() {
    Serial.begin(115200); // Khởi động Serial Monitor
    pinMode(LED_PIN, OUTPUT); // LED: OUTPUT    pinMode(BTN_PIN,
    INPUT_PULLUP); // Nút: INPUT với pull-up nội    dht.begin();
    // Khởi động cảm biến DHT22    display.setBrightness(0x0f); // Độ
    sáng tối đa (0x0f = 15)

    // Kết nối Blynk với Auth Token và thông tin Wi-Fi
    Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

    // Đặt lịch gọi hàm sendDataToBlynk mỗi 1000ms (1 giây)
    timer.setInterval(1000L, sendDataToBlynk);
    // Kiểm tra nút nhấn mỗi 100ms (tần số cao để không bỏ lỡ)
    timer.setInterval(100L, checkPhysicalButton);
}

void loop() {
    Blynk.run(); // Xử lý kết nối và sự kiện Blynk    timer.run();
    // Kích hoạt các hàm timer đã đặt lịch
}
```

3.3. Luồng xử lý dữ liệu

Luồng xử lý dữ liệu của hệ thống diễn ra theo chu kỳ liên tục sau:

8. ESP32 khởi động, kết nối Wi-Fi và xác thực với Blynk Cloud qua Auth Token.
9. BlynkTimer kích hoạt hàm checkPhysicalButton() mỗi 100ms để quét trạng thái nút nhấn.
10. BlynkTimer kích hoạt hàm sendDataToBlynk() mỗi 1000ms (1 giây).
11. DHT22 đo nhiệt độ và độ ẩm, dữ liệu được gửi lên V1 và V2 qua Blynk.virtualWrite().
12. Blynk Cloud nhận dữ liệu, lưu vào cơ sở dữ liệu và cập nhật Dashboard theo thời gian thực.
13. Khi người dùng tương tác với Switch trên Dashboard, lệnh được gửi xuống V0, callback BLYNK_WRITE(V0) thực thi và cập nhật trạng thái LED.

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. Hướng dẫn triển khai và kiểm thử

4.1.1. Chuẩn bị môi trường

14. Cài đặt Arduino IDE phiên bản 2.x từ <https://www.arduino.cc/en/software>.
15. Cài đặt ESP32 board package: Vào File → Preferences → Additional Board Manager URLs, thêm: https://dl.espressif.com/dl/package_esp32_index.json. Sau đó vào Tools → Board → Board Manager, tìm "esp32" và cài đặt phiên bản mới nhất.
16. Cài đặt thư viện: Vào Sketch → Include Library → Manage Libraries, lần lượt tìm và cài: "Blynk", "DHT sensor library" (của Adafruit), "TM1637".

4.1.2. Quy trình chạy mô phỏng trên Wokwi

17. Truy cập <https://wokwi.com> và tạo project mới với ESP32.
18. Dán nội dung file diagram.json vào tab "diagram.json" để tự động tạo sơ đồ mạch.
19. Dán mã nguồn vào file sketch.ino.
20. Thay thế giá trị BLYNK_AUTH_TOKEN bằng token thực của thiết bị đã tạo trên Blynk Cloud.
21. Nhấn nút Play (▶) để bắt đầu mô phỏng. Theo dõi Serial Monitor để xem log kết nối.
22. Khi thấy log "Ready" xuất hiện trên Serial Monitor, thiết bị đã kết nối thành công với Blynk.

4.1.3. Kiểm tra kết quả trên Blynk Dashboard

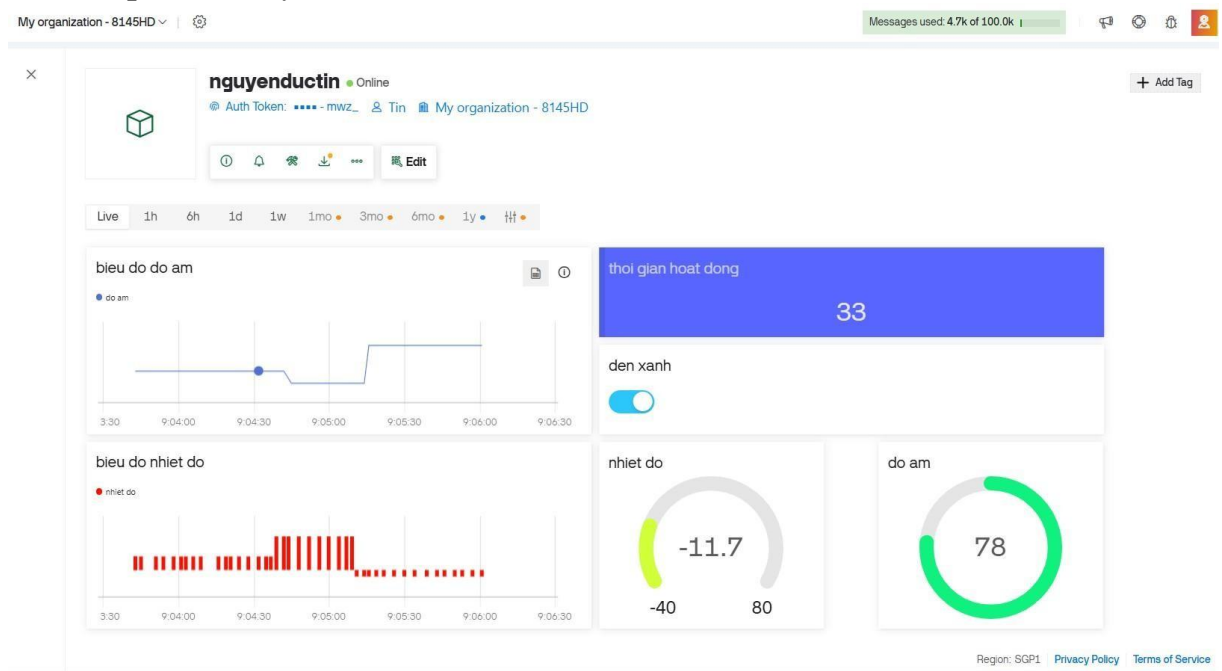
23. Mở Blynk Web Dashboard tại <https://blynk.cloud>.
24. Điều hướng đến thiết bị vừa tạo, quan sát trạng thái "Online" (chấm xanh).
25. Biểu đồ nhiệt độ và độ ẩm sẽ bắt đầu cập nhật mỗi giây.
26. Thử bật/tắt switch "den_xanh" trên Dashboard và quan sát LED thay đổi trong mô phỏng Wokwi.
27. Thử nhấn nút vật lý trên Wokwi và kiểm tra trạng thái switch trên Dashboard có cập nhật đồng bộ không.

4.2. Kết quả thực nghiệm

4.2.1. Kết quả mô phỏng Wokwi

Kết quả chạy mô phỏng trên Wokwi cho thấy hệ thống hoạt động đúng theo thiết kế. Màn hình TM1637 hiển thị giá trị uptime tăng dần theo từng giây khi LED đang bật. LED xanh phản hồi tức thì với cả hai phương thức điều khiển (nút nhấn vật lý và Blynk Dashboard). Cảm biến DHT22 trong môi trường mô phỏng trả về các giá trị ổn định theo thời gian thực.

4.2.2. Kết quả trên Blynk Dashboard



Hình 4.1: Giao diện Blynk Dashboard hiển thị dữ liệu thực tế

Kết quả thực nghiệm trên Blynk Dashboard (Hình 4.1) cho thấy:

- Widget "thời gian hoạt động" hiển thị giá trị 88 giây, phản ánh thời gian hoạt động liên tục của thiết bị.
- Gauge "nhiệt độ" hiển thị -11.7°C và gauge "độ ẩm" hiển thị 29.5% – đây là các giá trị mô phỏng từ Wokwi.
- Biểu đồ "biểu đồ độ ẩm" cho thấy độ ẩm ổn định trong khoảng thời gian quan sát, với sự thay đổi nhỏ tại 6:13 PM.
- Biểu đồ "biểu đồ nhiệt độ" cho thấy các điểm đo liên tục và dày đặc, chứng minh chu kỳ gửi dữ liệu 1 giây hoạt động hiệu quả.

- Switch "den xanh" ở trạng thái ON (màu xanh), đồng bộ với trạng thái LED trong mô phỏng.

4.3. Đánh giá hệ thống

4.3.1. Ưu điểm

- Kiến trúc đơn giản, chi phí thấp: ESP32 và DHT22 là các linh kiện phổ biến, dễ tìm với giá thành dưới 100.000 VNĐ.
- Khả năng giám sát từ xa: Blynk Cloud cho phép theo dõi và điều khiển từ bất kỳ đâu có kết nối internet.
- Dữ liệu lịch sử: Blynk tự động lưu trữ dữ liệu, hỗ trợ phân tích xu hướng.
- Đồng bộ hai chiều: Điều khiển từ cả Dashboard và nút vật lý với cơ chế đồng bộ trạng thái.
- Mã nguồn tối ưu: Sử dụng BlynkTimer thay vì delay() giúp hệ thống hoạt động non-blocking, không bị treo.

4.3.2. Hạn chế và hướng khắc phục

Hạn chế	Nguyên nhân	Hướng khắc phục
Phụ thuộc internet	Blynk yêu cầu kết nối liên tục	Tích hợp lưu trữ cục bộ (SD card, EEPROM) khi mất mạng
Blynk Free bị giới hạn	Plan miễn phí giới hạn thiết bị và datastream	Nâng cấp lên Blynk Plus hoặc triển khai Blynk Local Server
Bảo mật Auth Token	Token cứng trong code dễ lộ khi chia sẻ	Lưu token trong SPIFFS/NVS, không hardcode trong source
DHT22 tốc độ thấp	Chu kỳ tối thiểu 0.5 giây	Dùng SHT31 hoặc BME280 cho ứng dụng yêu cầu tần suất cao hơn

4.4. Hướng phát triển

Dựa trên nền tảng đã xây dựng, hệ thống có thể được mở rộng theo nhiều hướng:

- Tích hợp cảm biến bổ sung: CO2 (MH-Z19), ánh sáng (BH1750), áp suất (BMP280) để xây dựng trạm quan trắc môi trường đầy đủ.
- Cơ chế cảnh báo thông minh: Cấu hình Blynk Automation để gửi email/SMS khi nhiệt độ hoặc độ ẩm vượt ngưỡng an toàn.
- Lưu trữ dữ liệu cục bộ: Tích hợp thẻ nhớ SD hoặc SPIFFS để ghi log khi mất kết nối internet.
- Giao thức MQTT thuần túy: Chuyển sang MQTT broker (Mosquitto, HiveMQ) để tăng tính linh hoạt và giảm phụ thuộc vào vendor.
- Giao diện Web tự build: Phát triển Web server nội bộ trên ESP32 để giám sát qua LAN không cần internet.

KẾT LUẬN

Tiểu luận đã trình bày một cách có hệ thống và toàn diện quá trình thiết kế, lập trình và kiểm thử hệ thống giám sát nhiệt độ và độ ẩm dựa trên vi điều khiển ESP32 kết hợp với nền tảng Blynk Cloud.

Về mặt lý thuyết, tiểu luận đã cung cấp nền tảng kiến thức vững chắc về ba thành phần cốt lõi: vi điều khiển ESP32 với kiến trúc dual-core và tích hợp Wi-Fi; cảm biến DHT22 với giao thức truyền dữ liệu single-wire và độ chính xác cao; và nền tảng Blynk Cloud với cơ chế Virtual Pins, Auth Token và Web Dashboard linh hoạt.

Về mặt thực tiễn, hệ thống đã được hiện thực hóa hoàn toàn trên môi trường mô phỏng Wokwi với kết quả đáng tin cậy: dữ liệu nhiệt độ và độ ẩm được thu thập với chu kỳ 1 giây và hiển thị trực tiếp trên Blynk Dashboard; cơ chế điều khiển hai chiều (Dashboard và nút vật lý) hoạt động đồng bộ chính xác; màn hình TM1637 phản ánh thời gian hoạt động theo thời gian thực.

Qua quá trình thực hiện đề tài, có thể rút ra một số nhận định quan trọng: ESP32 là nền tảng phần cứng lý tưởng cho các ứng dụng IoT tầm trung nhờ sự kết hợp hoàn hảo giữa sức mạnh xử lý, kết nối không dây tích hợp và chi phí thấp. Blynk 2.0 cung cấp trải nghiệm phát triển IoT nhanh chóng với cơ sở hạ tầng đám mây sẵn sàng, phù hợp cho cả prototyping lẫn triển khai thực tế quy mô nhỏ. Mô hình kiến trúc phân lớp (thu thập – xử lý – hiển thị) đã được chứng minh hiệu quả và dễ mở rộng.

Hệ thống đã xây dựng có thể được áp dụng trực tiếp trong nhiều kịch bản thực tế: giám sát kho lạnh, nhà kính nông nghiệp, phòng server, hoặc các không gian cần kiểm soát vi khí hậu. Đây là minh chứng rõ ràng cho tiềm năng của công nghệ IoT trong việc số hóa và tự động hóa các quy trình quản lý môi trường truyền thống.

TÀI LIỆU THAM KHẢO

Tiếng Việt :

[1] Nguyễn Minh Tuấn (2022). Lập trình IoT với ESP32 và Arduino IDE. Nhà xuất bản Thông tin và Truyền thông, Hà Nội.

[2] Tham khảo tại Claude.ai .

Tiếng Anh :

[3] Espressif Systems (2023). ESP32 Datasheet Version 4.1. Espressif Systems. Truy cập tại:

https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

[4] AOSONG Electronics (2022). DHT22 Product Manual. Guangzhou AOSONG Electronics Co., Ltd.

Truy cập tại: <https://www.aosong.com/en/products-21.html>.

[5] Blynk Inc. (2024). Blynk Documentation – Getting Started with ESP32. Blynk.

Truy cập tại: <https://docs.blynk.io/en/getting-started/what-do-i-need-to-blynk>

[6] Espressif Systems (2023). ESP32 Technical Reference Manual. Espressif Systems. Truy cập tại:

https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.

[7] Wokwi Team (2024). Wokwi Simulator Documentation. Wokwi.

Truy cập tại: <https://docs.wokwi.com>.

[8] Adafruit Industries (2023). DHT Humidity & Temperature Sensors – Arduino Library. GitHub.

Truy cập tại: <https://github.com/adafruit/DHT-sensor-library>.

[9] Avishai Michaeli (2022). TM1637Display Library Documentation. GitHub.

Truy cập tại: <https://github.com/avishorp/TM1637>.

[10] Stanford CS348K (2022). IoT Architecture Patterns and Best Practices. Stanford University.

Truy cập tại: <https://gforcourses.stanford.edu/cs348k>.